

Python

▼ What is Python ?

→ Python is a **high-level, interpreted programming language** known for its simplicity and readability. It is widely used for web development, data science, artificial intelligence, machine learning, automation, scripting, and more. Python emphasizes code readability with its clean syntax and indentation-based structure.

Key Features of Python:

- **Interpreted** : Executes code line by line.
- **Dynamically Typed** : No need to declare variable types.
- **Extensive Libraries** : Has rich libraries like NumPy, Pandas, TensorFlow, Django, Flask, etc.

▼ Installation of Python ?

STEPS :

1. Go to the official Python website: <https://www.python.org/downloads/>
2. Click on "**Download Python [latest version]**" (e.g., Python 3.x.x).
3. The installer (`.exe` file) will be downloaded.
4. Open **Command Prompt (cmd)**:
5. Type `cmd` , and press **Enter**.
6. Type the following command and press **Enter** : If Python is installed correctly, it will display the installed version.

" python --version "

▼ What is Python Indentation ?

→ Indentation in Python refers to **the spaces or tabs at the beginning of a line of code** that define the structure of the program. Unlike other programming languages that use curly braces `{ }` to mark code blocks, Python **strictly enforces indentation** to indicate blocks of code.

→ The number of spaces is up to you as a programmer, the most common use is four, but it has to be at least one.

▼ Python Variables ?

→ A **variable** in Python is used to store data. It acts as a container that holds values, which can be changed during program execution.

▼ Declaring a Variable in Python :

- Python **automatically assigns** the data type based on the value.
- No need to specify `int` , `float` , `string` , etc.

```
x = 10 # Integer
y = "Sanjai" # String
z = 3.14 # Float
```

▼ Variable Names :

Rules for Naming Variables

✔ Valid Variable Names:

1. Must start with a **letter (a-z, A-Z) or an underscore** `_`, but not a number.
2. Can contain **letters, numbers (0-9), and underscores** `_`.
3. Variable names are **case-sensitive** (`age` and `Age` are different).
4. Should not use **Python keywords** (e.g., `if`, `else`, `class`, `def`, etc.).

```
name = "Sanjai" # Valid
_age = 22 # Valid
totalAmount = 500 # Valid
```

Best Practices for Naming Variables

- Use **descriptive** names (`student_name` instead of `s`).
- Follow **snake_case** (`first_name`, `total_amount`).
- Use **camelCase** if working with JavaScript-like syntax (`firstName`).

▼ ***Assign Multiple Values :***

→ Python allows assigning **multiple values** to multiple variables in one line.

Example : Assign Different Values

```
a, b, c = 10, 20, 30
print(a, b, c) # Output: 10 20 30
```

Example : Assign Same Value to Multiple Variables

```
x = y = z = 100
print(x, y, z) # Output: 100 100 100
```

Example : Unpacking a List or Tuple

```
fruits = ["apple", "banana", "cherry"]
x, y, z = fruits
print(x, y, z) # Output: apple banana cherry
```

▼ ***Output Variables :***

→ You can display the value of variables using `print()` .

Example : Printing Variables

```
name = "Sanjai"
age = 22
print(name)
print(age)
```

Example : Combining Strings and Variables

```
name = "Sanjai"
print("Hello, " + name) # Output: Hello, Sanjai
```

Example : Using `f-strings` (Recommended)

```
age = 22
print(f"I am {age} years old.") # Output: I am 22 years old.
```

Example : Using `format()`

```
name = "Sanjai"
age = 25
print("My name is {} and I am {} years old.".format(name, age))
```

▼ **Global Variables :**

→ A **global variable** is a variable declared outside of a function and can be accessed anywhere in the program.

Example : Global Variable

```
x = "Python" # Global variable

def my_function():
    print("I love " + x)

my_function() # Output: I love Python
```

Using the `global` Keyword

→ To modify a **global variable** inside a function, use the `global` keyword.

```
x = "Python" # Global variable

def my_function():
    global x
    x = "JavaScript" # Modify global variable

my_function()
print(x) # Output: JavaScript
```

▼ **Python Data Types ?**

→ A **data type** defines the type of value a variable holds. Python provides several built-in data types.

▼ **Types of Data Types :**

Text Type	<code>str</code>
Numeric Types	<code>int</code> , <code>float</code> , <code>complex</code>
Sequence Types	<code>list</code> , <code>tuple</code> , <code>range</code>
Mapping Type	<code>dict</code>
Set Types	<code>set</code> , <code>frozenset</code>
Boolean Type	<code>bool</code>
Binary Types	<code>bytes</code> , <code>bytearray</code> , <code>memoryview</code>
None Type	<code>NoneType</code>

▼ **Setting the Specific Data Type :**

Example	Data Type	OutPut
<code>x = str("Sanjai Kannan G")</code>	<code>str</code>	Sanjai Kannan G - str
<code>x = int(22)</code>	<code>int</code>	22 - int

x = float(22.5)	float	22.5 - float
x = complex(1j)	complex	1j - complex
x = list(("sanjai", "kannan", "gokul"))	list	["sanjai", "kannan", "gokul"] - list
x = tuple(("sanjai", "kannan", "gokul"))	tuple	("sanjai", "kannan", "gokul") - tuple
x = range(6)	range	range(0, 6) - range
x = dict("name" : "sanjai", "age" : 22)	dict	{'name': 'sanjai', 'age': 22} - dict
x = set(("apple", "banana", "cherry"))	set	{'apple', 'banana', 'cherry'} - set
x = frozenset(("apple", "banana", "cherry"))	frozenset	frozenset({'banana', 'apple', 'cherry'})
x = bool(5)	bool	True
x = bytes(5)	bytes	b'\x00\x00\x00\x00\x00'
x = bytearray(5)	bytearray	bytearray(b'\x00\x00\x00\x00\x00')
x = memoryview(bytes(5))	memoryview	<memory at 0x00D58FA0>

▼ **Python Casting (Type Conversion) ?**

→ Python allows you to convert data from one type to another using **casting functions**.

1. Casting to Integer (`int`) :

→ Converts values to an integer (removes decimals, converts strings if valid).

```
x = int(3.9)    # 3
y = int("10")   # 10
z = int(True)   # 1
```

2. Casting to Float (`float`) :

→ Converts values to a floating-point number.

```
x = float(5)    # 5.0
y = float("3.14") # 3.14
z = float(False) # 0.0
```

3. Casting to String (`str`) :

→ Converts values to a string.

```
x = str(100)    # "100"
y = str(3.14)   # "3.14"
z = str(True)   # "True"
```

4. Casting to Boolean (`bool`) :

→ Converts values to `True` or `False` .

- `0` , `None` , `""` , `[]` , `{}` → **False**
- Everything else → **True**

```
x = bool(0)     # False
y = bool(10)    # True
z = bool("Hello") # True
```

5. Casting to List (`list`) :

→ Converts an iterable (tuple, string, range) to a list.

```
x = list("hello")    # ['h', 'e', 'l', 'l', 'o']
y = list((1, 2, 3))  # [1, 2, 3]
z = list(range(3))   # [0, 1, 2]
```

6. Casting to Tuple (`tuple`) :

→ Converts an iterable to a tuple.

```
x = tuple([1, 2, 3]) # (1, 2, 3)
y = tuple("hello")   # ('h', 'e', 'l', 'l', 'o')
```

7. Casting to Set (`set`) :

→ Converts an iterable to a set (removes duplicates).

```
python
CopyEdit
x = set([1, 2, 2, 3]) # {1, 2, 3}
y = set("banana")    # {'a', 'b', 'n'}
```

8. Casting to Dictionary (`dict`) :

→ Converts a list of key-value pairs to a dictionary.

```
x = dict([("name", "Sanjai"), ("age", 25)])
# {'name': 'Sanjai', 'age': 25}
```

▼ Python Strings ?

▼ Strings :

→ A **string** in Python is a sequence of characters enclosed in **single**, **double**, or **triple quotes**.

```
print("Sanjai") #Double Quotes
print('Sanjai') #Single Quotes
```

▼ Slicing Strings :

→ Python allows **indexing** and **slicing** strings using `[]`.

```
text = "Sanjai Kannan G"

# Accessing characters
print(text[0])  # S (first character)
print(text[-1]) # G (last character)

# Slicing [start:end:step]
print(text[0:6]) # Python (0 to 5)
print(text[7:])  # Kannan G (from index 7 to end)
print(text[:6])  # Sanja (start to index 5)
print(text[::2]) # Sna annG (every second character)
print(text[::-1]) # G nannaK iajnaS (Reverse string)
```

▼ **Modify Strings :**

→ Python provides multiple string modification methods.

```
text = " Sanjai Kannan G"

# Changing case
print(text.upper())  # " HELLO WORLD "
print(text.lower())  # " hello world "
print(text.title())  # " Hello World "
print(text.capitalize()) # " Hello world "

# Removing spaces
print(text.strip())  # "hello world"
print(text.lstrip()) # "hello world " (removes left spaces)
print(text.rstrip()) # " hello world" (removes right spaces)

# Replacing
print(text.replace("hello", "Hi")) # " Hi world "
```